Assignment 3 - Sustainable software engineering

ADNET Willem (12444262), Klemens Hammerer (12211961), and ZABIEGO Hugo (12445574)
group I,

18/05/2025

Abstract

Since around 2010, the term Sustainable Software Engineering has emerged, bringing with it new perspectives on how software should be designed and developed. Sustainable software engineering emphasizes the importance of considering energy consumption—and more generally, resource usage—throughout the software development process. Today, when we develop new solutions to computational problems, we aim to optimize not only for execution time but also for energy efficiency.

1 Introduction

To improve the way we program by reducing execution time and energy consumption, it is essential to measure energy usage accurately. Measurement is the only way to determine whether our code modifications actually lead to more sustainable software. To do this, we used several tools, depending on the operating system:

- macOS: `/usr/bin/time -l` or `powermetrics`
- Linux: `powerstat`, `powertop`, or `perf`
- Windows: `Intel Power Gadget`

These tools often rely on hardware-specific capabilities. For example, Apple Silicon chips present additional challenges, as their energy consumption data is less accessible compared to other systems, where such data can be more easily retrieved. That is why we used different programs for different operating systems / hardware.

2 Energy tests on different sum computations

First, we created a program that calculates different sums of the values in an array sequentially :
$\frac{1}{n^k} \times \sum_{i=0}^{n} X_i^k, k = 1, .., 5$ with n the length of the array under study.

We then made two programs with different levels of optimization. The less optimized program calculates each sum one by one and therefore uses many loops to reach the final result. The optimized program, on the other hand, uses a single loop to calculate and add the different sums. One loop thus performs the calculations for all the sums, which helps reduce complexity. Additionally, some powers are sometimes also optimized to avoid recalculating them.

We then moved on to the testing phase and noticed differences in energy consumption: the Cumulative Processor Energy indicates an average of 215 Joules for the non-optimized program and around 160 Joules for the optimized one (test performed using the Powerstat tool). Furthermore, with the second tool, Perf, we obtained an average of 80,000 Joules versus 55,000 Joules for the energy-cores category, and 155,000 versus 117,000 for energy-psys. We also observed that all energy-related categories are affected by the difference in programming, which shows that program optimization is effective not only at the CPU level, but also in terms of RAM, GPU, and other components.

Metric	Non optimized	Optimized
Energy Cores [Joules]	78,24	56,15
Energy RAM[Joules]	6,01	4,48
Energy GPU[Joules]	0,89	0,69
Energy Psys [Joules]	155,5	118,7

Table 1: Results of the measurements

3 Experimental Setup and Observations (Apple Silicon)

The goal of this experiment was to understand and quantify the energy consumption of software by comparing two versions of a program: one deliberately inefficient (energy-intensive) and one optimized. We focused on observing the impact of computational complexity and unnecessary operations on power usage.

3.1 Program Description

We implemented a heavy version of the Fibonacci sequence generator in Java. The algorithm includes recursive calls and adds a costly mathematical computation inside the recursion using trigonometric and square root functions:

- The method `expo_fibo` adds artificial CPU load.
- The `run` method recursively calculates Fibonacci values, compounding the load.
- The main thread parallelizes execution over several Fibonacci numbers using a fixed thread pool.

The goal of this version was to simulate a high CPU load and observe its impact on energy usage.

3.2 Measurement tools

We used different tools like:

- `/usr/bin/time -l` was used to get resource usage including CPU time and memory footprint.
- `powermetrics` was used to attempt direct energy readings.

The `powermetrics` analysis was performed using the Python script `analyse_data_efficient.py`, which processes the report generated by the Bash script `measure_energie.sh`.

3.3 Results and discussion

The results showed into the results files : `heavyConsumption.md` compare to the file : `lightConsumption.md` shows clearly that adding some floating points calculations, not reusing previous results and increasing the number of threads does increase the consumption of energy. As a sum up we do have :

- More floating-point operations increased energy use.
- Parallel threads increased performance but also power draw.
- Rewriting the program to avoid deep recursion and unnecessary math led to shorter execution time and lower energy use.

As a matter of number the results are the next ones :

Metric	**Heavy Version**	**Light Version**
Execution Time (s)	23	1
CPU Usage (mW)	4939	3229
GPU Usage (mW)	8	22
Total Usage (mW)	4946	3251
Energy (J)	113	3.251

Table 2: Results of the measurements

4 Memory accesses and Types

In this part of the task we used a program that just writes every second value counting up to storage and then in a second loop we display it.

4.1 Program Description

In the non optimized part the program iterates from 0 to the defined max value. A bool that is just switched every iteration keeps track if we are on a odd or even number. If the number is even the number is added to a ArrayList of integers. In the printing phase again every other number is skipped and we then read the wanted value from the list and print it.

The optimizations for the optimized program remove the need for the bool and just increase the iteration variable by two. With that 2 ifs are removed. Same for the printing phase. The other optimization was to use an array instead of a list. This has the benefit that the memory access is constant so writing and reading takes place in the same time if it's the first or last element. In a list reading takes longer the longer the list gets.

4.2 Measurement tools

In this part we used `Intel Power Gadget` as the software to read power usage. As well as the power usage other important information is given by the program like the cpu frequency, thermal values and also elapsed time. All these values help better understand what really is happening with your code and if it's getting more sustainable or not.

4.3 Results and discussion

To get more reliable Values the programs were each run 3 times. The following values are the averages of the 3 runs.

As can be seen in Table 3 the optimized code uses much fewer resources to execute the code while achieving the same output. This is mainly due to the execution time. The non optimized code took about 10 seconds to execute. the optimized one took 3,5. If you take any other value of the optimized coloumn and multiply it by about three - as the optimized version was about 3 times as fast – you'll get comparable values. The execution time is influenced by the loops but also by the memory accesses. Since you have to iterate through the list to get to later values the execution time gets longer as the list gets bigger. This isn't as big a problem with the limit we set in this example but when we tested it with higher limits the execution time grew fast.

Metric	Non optimized	Optimized
Energy [Joules]	306,2	89,22
Energy [mWh]	85	24,08
DRAM Energy [Joules]	13,95	4,45
DRAM Energy [mWh]	3,87	1,26
Elapsed Time [s]	10,27	3,5

Table 3: Results of the measurements

5 Reflection on energy measurement

Measuring energy consumption in software is complex and hardware-dependent. However, relative comparisons (same machine, repeated runs) provide reliable insight. The most effective way to measure software sustainability is:

- Measure CPU time and power draw using OS-specific tools.
- Repeat runs to smooth out variations.
- Focus on relative differences between program versions rather than absolute values.

6 Conclusion

The goal of this task was to learn about sustainability and energy consumption in combination with software development. How does our code affect execution time, energy consumption and how would someone optimize it.

The most obvious improvement one can make to it's code is to shorten the execution time. Skipping unnecessary code segments (calculations) / iterations in loops is an example for that. Also improving your algorithms can have an effect on energy consumption. Another improvement is data storage. Depending on your use an array maybe could do the job better than a list. Since accesses to arrays are faster this could lower execution time and with that energy consumption. Last but not least different operations have different complexity and use different amounts of energy. An example for that would be the pow function. It is really expensive to do that operation so maybe think about other ways to calculate the same result. For example if you square a value, just multiply it with itself instead of using the pow function.